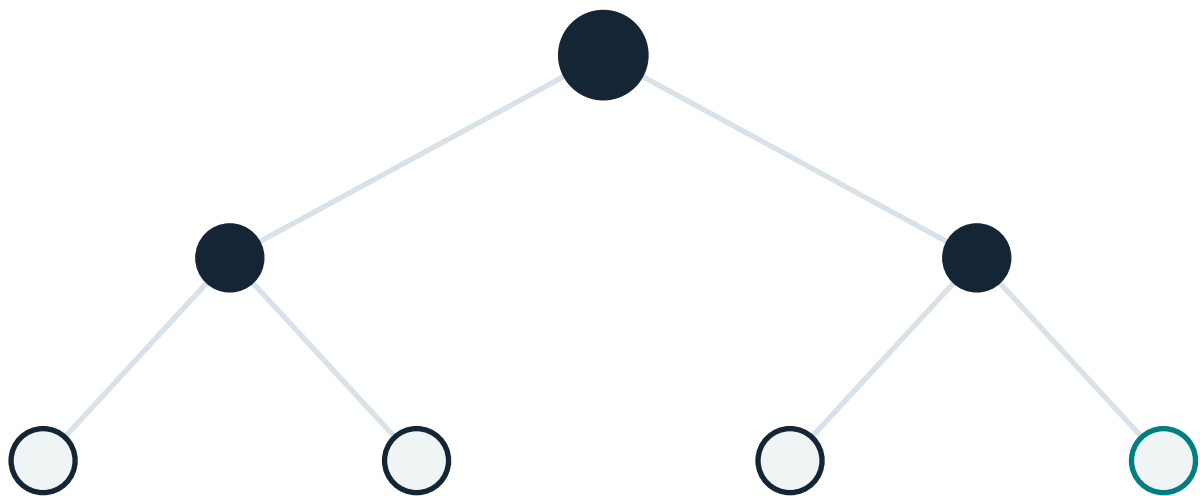

Pactra

System Architecture & User Workflows

One budget for a tree of agents, enforced on chain.



PROJECT MAINTAINER

SomeshTalligeriDEV

DOCUMENT CONTROL

Version 1.0 / 17 September 2026

ASSESSMENT

Source-reviewed architecture; local rebrand

Executive overview

Pactra bounds the autonomous spending of a delegated agent tree. A human opens and funds a root mandate. Each descendant may receive narrower authority, while every accepted draw consumes the budget of that node and every ancestor. This prevents independently compliant workers from collectively bypassing their shared root budget.

Core invariant: an autonomous draw is permitted only when every mandate on the path to the root permits it and the root treasury can fund it. Explicit owner releases are separately recorded exceptions.

Stakeholder	Responsibility
Owner	Open and fund mandates; inspect spending; release a refusal, revoke a branch, or withdraw funds.
Operator / developer	Run the key-holding runtime, fund operator gas, configure integrations, and maintain deployment services.
Agent	Request a URL, inspect headroom, or request a narrower child; receives no private key through these interfaces.
Seller / public reader	Sell an x402 resource or inspect public, chain-linked conduct information.

Assessment boundary

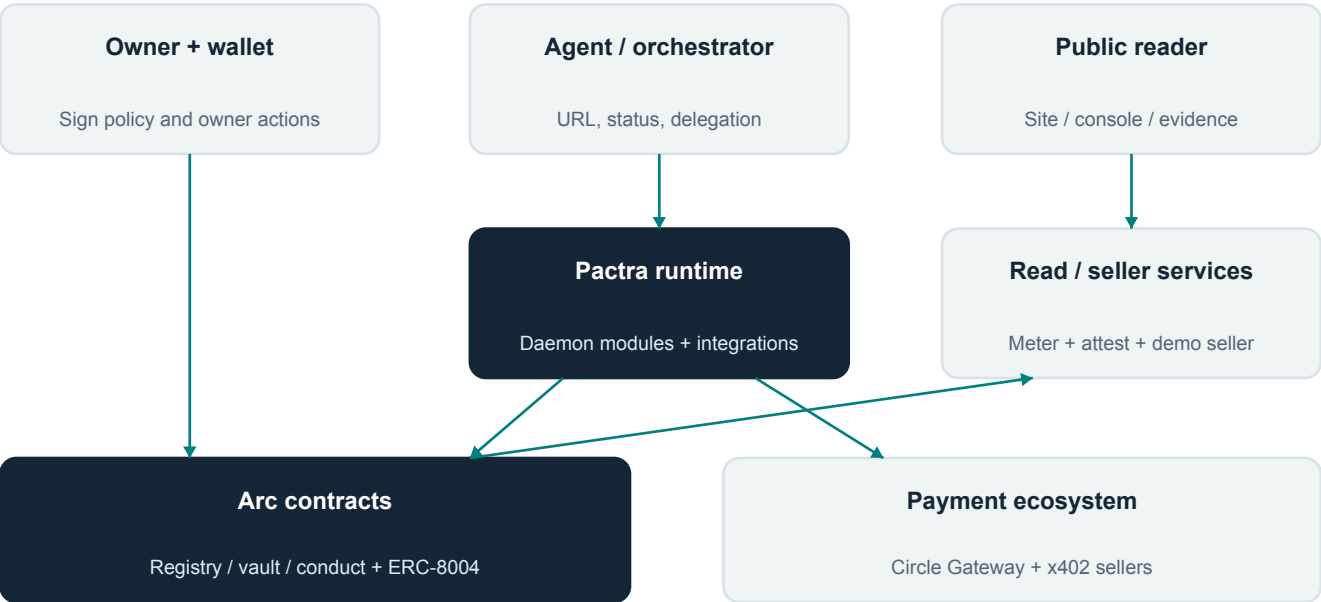
This is an as-built architecture and workflow description of the supplied filesystem, reviewed on 17 September 2026. It uses the implementation as primary evidence. Existing deployment addresses and recorded benchmark results are historical artifacts; they were not revalidated against public services for this document. This is not a certification or a completed security audit.

The system is an Arc testnet implementation. The rename updates local application and operational source. It does not change a deployed contract, publish a package, acquire a domain, or provision a repository. Pactra deployment examples use the reserved placeholder pactra.example.

Reading guide

Sections 02–05 describe structure and enforcement. Sections 06–10 follow purchases and user journeys. Sections 11–13 cover APIs, data, and operations. Sections 14–16 record limitations, migration decisions, and validation. Section 17 maps the document to source files.

System context



Pactra spans a human-controlled browser, a private agent runtime, a public read-and-sell host, and external blockchain/payment services. Authority and information follow different paths: owner actions go directly to contracts, agent purchases go through the key-holding runtime, and public pages consume indexed evidence.

Boundary	What crosses it
Browser → chain	Wallet-signed contract transactions; the browser does not obtain daemon operator keys.
Agent → runtime	Tool calls or HTTP requests. The HTTP variant selects a node supplied by the caller; MCP binds the session to one configured node.
Runtime → seller / rail	URL requests, seller challenges, Gateway operations, and a signed payment authorization.
Chain → public services	Events and view calls used to build readable conduct records; these caches do not authorize spending.

External dependencies are Arc RPC and USDC, Circle Gateway, ERC-8004 identity and reputation registries, seller endpoints, and Privy for the configured wallet experience. Their availability and semantics are outside Pactra's control.

03 / Container and code view

Components and responsibilities

Package	Role and implementation
contracts	Solidity 0.8.28; MandateRegistry, TreeVault and ConductRecord. Foundry builds and tests; Shanghai EVM target.
daemon	Node / TypeScript / viem. Configuration, key storage, serialized chain operations, challenge selection, payment settlement, publication, and HTTP API.
mcp	MCP SDK / Zod. Stdio tool server that uses daemon modules in process; a separate HTTP daemon is not required for this integration.
proxy	Node HTTP/TLS adapter and command wrapper. Routes compatible programs through the HTTP daemon; optional ephemeral CA for HTTPS inspection.
meter	Node / viem event indexer, in-memory ledger, atomic JSON snapshots, read-only API, and limited Gateway reconciliation.
attest	x402 seller of conduct snapshots, plus a demo seller entry point. Runs its own ledger sync; submits payer-signed collection transactions.
console	React 18 / Vite / React Router / Privy / viem. Overview, agent tree, refusal management, mandate creation, and owner transactions.
site	React / Vite documentation, product pages, drill evidence, and public agent / refusal / attestation pages.
ui + brand	Shared React components, styles and primitives; generated SVG/PNG wordmarks, icons, manifests and sharing assets.
fixtures + eval	Shared amounts, addresses, schemas, generated evidence and examples; deterministic evaluation harness and local-chain scenarios.

Dependency shape

There is no root workspace package manager configuration in the supplied copy. Packages have individual manifests and, where applicable, npm lockfiles. Backend modules often import sibling TypeScript files directly. Frontends resolve pactra-ui and @pactra/fixtures through TypeScript and Vite aliases.

The public attest HTTP service is distinct from ConductRecord.attest(): the service sells a reading; the Solidity function publishes a refusal to a reputation registry.

Contract domain and authority

Object	Identity, state and authority
Mandate	bytes32 node ID; parent/root links; immutable owner/operator and spending parameters; depth and revocation flag. Root IDs include chain, registry, owner and nonce.
Root treasury	USDC balance tracked per root by TreeVault.treasury6. Anyone may fund a root; only its owner may withdraw.
Budget counters	Per-node window spend, lifetime spend, and per-declared-counterparty window spend. Successful draws update all ancestors.
Refusal	One-based ID within a vault. Stores attempted node, breached ancestor, declared payee, amount, reason, timestamp and released flag.
Identity binding	ConductRecord maps a node to an ERC-8004 identity and back. Binding checks that the identity belongs to the node's operator.
Published feedback	Derived from the vault refusal rather than caller-supplied content. Tracks whether original and release records have been attested.

Who can change what

The owner opens a root. The owner or parent operator may spawn a child, subject to narrowing. The owner or a strict ancestor's operator may revoke a node; descendants become non-live through ancestor checks. Only the operator named by a node can draw for it. Only the inherited owner can release that node's refusal.

No application administrator, upgrade proxy, pause controller, or mutable mandate editor exists in these three contracts. Replacing a policy requires a new mandate. Revocation affects future checks; it does not recall funds already released from the vault.

Owner releases are exceptions, not autonomous draws. `release()` bypasses normal budget evaluation, consumes treasury, and emits a separate event without incrementing window or lifetime draw counters. The original refusal remains.

05 / Invariants and calculations

Budget enforcement semantics

All spending amounts use six-decimal USDC base units and integer arithmetic. One USDC is 1,000,000 units. Basis-point concentration limits use a denominator of 10,000. Native-chain gas units and ERC-20 spending units must remain distinct.

Constraint	Implemented rule
Child narrowing	Child budget, lifetime cap, tranche cap and concentration cannot exceed its parent's. Window duration must equal the parent's; maximum depth is inherited.
Revocation	Reject a draw if the node or any ancestor is revoked.
Tranche	For every ancestor a: $\text{amount6} \leq \text{a.trancheCap6}$.
Window	For every ancestor a: $\text{currentWindowSpent(a)} + \text{amount6} \leq \text{a.budget6}$.
Lifetime	For every ancestor a: $\text{lifetimeSpent(a)} + \text{amount6} \leq \text{a.lifetimeCap6}$.
Concentration	$\text{DeclaredPayeeSpent(a, payee)} + \text{amount6} \leq \text{floor(a.budget6} \times \text{a.concentrationBps} / 10,000)$.
Liquidity	The root treasury must contain at least amount6.

Evaluate first; commit second

TreeVault evaluates the entire ancestry before committing counters and depositing the tranche to Gateway for the node's operator. A policy refusal writes a refusal and returns false. Invalid authorization or malformed inputs revert. A reverted downstream transfer also rolls back that transaction.

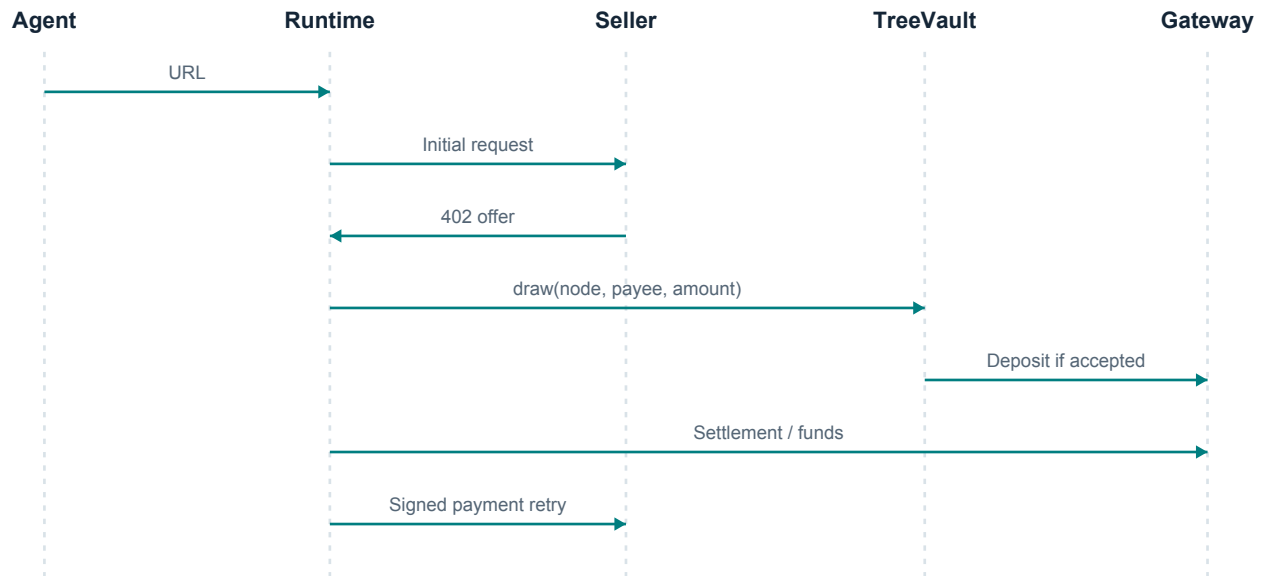
Window and headroom details

The actual implementation uses tumbling windows advanced in whole-duration steps, not a sliding trailing window. The UI's headroom is the minimum remaining window/lifetime authority across ancestors, capped by treasury and zeroed by revocation. Headroom is not a guarantee that one purchase will pass: tranche and seller-specific concentration still apply.

Example: a root permits 10 USDC per window and four children each permit 5. After two children draw 5 each, the root has no headroom and the other two cannot draw, despite their unused local budgets. Other constraints are assumed permissive in this illustration.

06 / Runtime sequence

One purchase, end to end



Simplified accepted path; the separate refusal and release branches are described below.

1. The agent supplies a URL. The runtime fetches it after egress checks. A response without a recognized payment challenge returns as unpaid/free; that classification does not by itself mean the HTTP request succeeded.
2. For a recognized 402 challenge, the runtime selects a supported network/asset offer and reads the price and payee from the seller. A matching, unspent owner release can fund the purchase; otherwise the runtime asks `TreeVault.draw()`.
3. Gate simulates and submits a transaction, then decodes its receipt. On acceptance, the vault debits every ancestor and deposits to the operator's Gateway balance. Settlement obtains usable funds through the configured Gateway route and signs an EIP-3009 authorization.
4. The runtime retries the seller request with `PAYMENT-SIGNATURE` and returns the response. The seller submits collection. A refusal instead returns structured refusal information; record publication is a separate operation and can fail.

A successful draw and a successful purchase are different milestones. The flow is not one atomic transaction across vault, Gateway and seller. The fetch result's paid flag indicates the payment path was attempted; inspect the returned seller status and settlement evidence before claiming delivery.

Owner onboarding and funding

Step / surface	Action and observable outcome
1. Prepare operators	Developer runs the daemon init command. It creates a protected key file and prints public operator addresses. Existing key files are not silently overwritten.
2. Open console	Owner visits /console. Without a Privy app ID the wallet layer is preview-only; configuring VITE_PRIVY_APP_ID enables the signing experience.
3. Create mandate	At /console/new, enter operator, window budget, lifetime cap, duration, tranche limit, concentration and maximum depth. UI rejects the owner's address as the operator.
4. Confirm transaction	The browser simulates and asks the wallet to call MandateRegistry.open. It reads the emitted node ID from the receipt rather than inventing one.
5. Fund treasury	Approve USDC as needed, then call TreeVault.fund. Opening a mandate alone does not deposit money.
6. Configure runtime	Set the deployment, node/key mapping, accepted assets and networks, and key-file path. Supply operator gas and start the daemon or MCP runtime.
7. Confirm readiness	Verify the live tree, treasury, node liveness, available headroom, operator gas, and identity enrollment before a paid request.

Feedback and recovery

Wallet rejection leaves the operation unsigned. Simulation errors should be resolved before paying transaction gas. After an uncertain transaction outcome, inspect its receipt before submitting again. A missing meter response is a read-service problem; the contract remains the authority.

Changing a policy

Bounds and operator assignments are immutable. To replace a root policy, revoke the old root, withdraw the remaining treasury to the owner's wallet, open a new mandate, fund it, and reconnect agents to the new node IDs. Existing records remain associated with the old deployment and mandates.

Source: console/src/screens/NewMandate.tsx, console/src/lib/mandate.ts, daemon/src/init.ts and daemon/src/config.ts.

Agent integration and delegation

Integration	Setup and scope
MCP	Run pactra-mcp from the checked-out source or a locally built bundle. Configure one node and its operator runtime. Tools are pactra_fetch, pactra_spawn and pactra_status.
HTTP	Run the daemon on port 8402. GET /status reads configured nodes; POST /fetch and POST /spawn name the acting node. Isolate this surface: it has no request authentication.
Proxy / wrapper	Configure PACTRA_NODE and PACTRA_DAEMON. The proxy defaults to port 8403. pactra-run starts a compatible program with proxy settings; HTTPS requires its scoped ephemeral CA.

Delegation workflow

A parent requests a child and specifies narrower authority plus an optional purpose label. The runtime generates the operator key and persists it before submitting the spawn. The registry checks parent authority, liveness, depth, equal window length, and narrowing. On success the runtime records the returned child ID beside the key.

The newly generated operator needs gas. Identity enrollment and purpose publication may therefore be incomplete even after a successful spawn. The HTTP response exposes this condition. Fund the public operator address and restart the runtime to retry enrollment and remembered-purpose publication.

Creating a mandate does not launch a child agent process. The orchestrator must start that process and configure its MCP/proxy integration to the child node. Purpose labels are descriptions, not enforced service-category restrictions.

Behavior on refusal

Read the refusal reason and breached ancestor. Do not repeatedly retry an unchanged request: refusals consume no spending budget, but transaction gas and publication attempts still have costs. Continue with a cheaper task, a different permissible resource, or an owner-reviewed exception.

The proxy is an integration convenience, not a network sandbox. Programs must honor proxy settings. Pactra's spend boundary assumes agents cannot obtain operator keys or an independent source of spendable funds.

09 / User journey C

Refusal handling and owner control

A policy rejection emits Refused and stores a durable tuple. It is returned as a normal structured outcome by the daemon, including over HTTP 200. The public meter indexes the event; the console presents the attempted node and the ancestor whose bound blocked it.

Owner decision	State transition and consequence
Leave refused	No autonomous funds move. The refusal remains visible. The task may change or wait for a relevant window to roll.
Release once	Owner signs <code>TreeVault.release(refusalId)</code> . Treasury funds move to the node operator's Gateway balance; Released is appended. No policy is widened and no refusal is deleted.
Cut a branch	Owner calls <code>MandateRegistry.revoke(node)</code> . Future normal draws and child spawns under the branch fail liveness checks.
Withdraw / replace	Owner withdraws unspent root treasury and, if desired, sets up a new mandate with different immutable bounds.

How the released purchase resumes

ChainReleases scans up to the newest 256 refusals by default and matches node, payee and exact amount. It checks current node liveness, reserves the matching ID in released-spent.json before settlement, and pays without submitting a fresh draw. A settlement failure can unclaim the reservation.

A crash after reservation may leave an unpaid release marked spent. The record prevents routine replay across restarts, but it is not a distributed transaction journal or a safe multi-process coordination mechanism. Recovery requires inspecting chain and seller outcomes, not blindly deleting the file.

Publication is distinct from enforcement

ConductRecord.attest derives feedback from the stored refusal. A later attestRelease appends evidence of the owner's exception. Missing identity binding, gas, RPC availability or publication execution can leave the refusal on chain but absent from the external registry. Releasing a refusal does not erase the initial feedback.

Source: `TreeVault.release`; `ConductRecord.attest/attestRelease`; `daemon/src/released.ts`; `console/src/screens/Refusals.tsx`.

Public evidence and paid attestations

Public reader

A visitor can browse the site without a wallet, read /docs, inspect /drill, open /agent/:id, or follow /refusal/:id. These surfaces combine generated examples with configured live meter reads. Preview content must be distinguished from records of an actual chain transaction.

Buyer of a conduct snapshot

Step	Service behavior
Discover	GET the attest service's /health to inspect price, asset, payee, network and indexed block range.
Request	GET /attest/:agentId. An unknown identity returns 404 before a payment challenge is offered.
Challenge	A known identity with no valid payment receives an x402 402 offer for the configured resource.
Pay	The buyer submits an authorization in PAYMENT-SIGNATURE or X-PAYMENT. The service validates terms and signature and submits collection through its collector.
Read	The buyer receives the conduct snapshot and provenance fields for the indexed range; a snapshot is evidence at that range, not a future guarantee.

Demonstration seller

The attest package also contains a demo seller for an Arc snapshot resource at /arc/snapshot. Operations configuration runs it separately on port 8406, while the conduct seller uses 8405. Source fixtures describe a one-cent conduct resource and a one-USDC demo snapshot; those are repository values, not independently checked live prices.

Three objects must remain distinct: a vault refusal, a published reputation record derived from it, and a purchased snapshot of indexed conduct. A paid snapshot does not create or authorize a refusal.

Historical drill and gate fixtures are evidence files supplied with the project. Renaming their surrounding labels does not rerun the experiments or prove that a new Pactra deployment achieved those results.

11 / APIs and tools

Interface contract reference

Surface	Interface	Meaning
Daemon	GET /status	Configured nodes, mandates and headroom.
Daemon	POST /fetch	node + url; optional method, headers and body. Seller determines payment terms.
Daemon	POST /spawn	Parent node and narrower policy; returns child/operator and recovery information.
MCP	pactra_fetch	URL and optional method/body for the fixed session node.
MCP	pactra_spawn	Label and budget; optional tranche/concentration; inherits other applicable bounds.
MCP	pactra_status	Aggregate headroom and binding ancestor.
Meter	GET /health	Index range and row counts.
Meter	GET /tree/:root	Subtree, funding/withdrawal totals and explicit truncation information.
Meter	GET /node/:node; /agent/:id	Conduct viewed by mandate or ERC-8004 ID.
Meter	GET /refusals; /refusal/:id	List supports root, limit and before; nextBefore is the continuation cursor.
Meter	GET /reconcile	Balance-bound check; 503 until a report exists.
Attest	GET /attest/:agentId	x402-protected conduct reading; health is unpaid.

Response and serialization rules

HTTP big integers are encoded as decimal strings. Snapshot files use a separate bigint representation with an n suffix. Clients must preserve integer precision. Meter answers expose chain and block-range information on the principal data responses, and callers must respect pagination and truncation.

Daemon policy refusal is a 200 response carrying refusal data, not transport success for a completed purchase. Seller challenges use 402; unknown resources use 404; unsupported read-API methods use 405. Do not treat every non-error transport status as proof of settlement.

These APIs are implemented directly with Node HTTP. There is no central OpenAPI contract or versioned API gateway in this copy. Tool schemas and generated fixtures provide part of the interface documentation.

12 / Persistence and evidence

Data lifecycle and consistency

Store	Contents and recovery model
Arc contract storage / events	Mandates, treasury, draw counters, refusals, releases and identity feedback. Financial authority is on chain.
Meter ledger + JSON snapshot	Reduced events and conduct aggregates. Incremental catch-up resumes after toBlock; write-temp-then-rename protects snapshot integrity.
Attest ledger	Independent chain-derived reading used by the seller; its indexed range can differ from the meter's.
Operator key file	Private operator keys, node mappings and remembered purpose labels. Generated with restrictive permissions; not reconstructible from chain.
released-spent.json	Local reservation/spend IDs for owner releases. Preserve alongside runtime state; loss can enable replay attempts and ambiguous recovery.
Fixtures and deployment files	Shared constants and historical generated evidence. Deployment addresses, block, and date are retained; the copied Git commit identifier was cleared.

Read consistency

The indexer reads events in bounded chunks, reduces into a new ledger, writes a snapshot and continues polling. A failed pass retains the last good ledger. Reconciliation can retain its last successful result independently, so a healthy process can still serve stale evidence.

The implementation is forward-only. Sync has an optional confirmation offset, but the production entry point does not provide a complete reorganization rollback workflow. Migration to a different finality model needs explicit confirmation, rollback and replay design.

What is not implemented

No PostgreSQL database or database migration layer is present, despite an old source comment naming Postgres as a possible cache. There is no end-to-end payment commitment journal, general idempotency key, or atomic transaction spanning draw, settlement and seller delivery.

Back up keys and local release state securely. Rebuild read caches from the deployment block when necessary; never treat deleting a financial recovery file as equivalent to rebuilding an index.

13 / Runtime topology

Deployment and operational model



Service	Default and configuration
Site + console	Static Vite bundles via nginx; console base /console/. VITE_PRIVY_APP_ID enables wallet signing; VITE_METER_URL selects live reads.
Meter	127.0.0.1:8404; PACTRA_RPC, chain/chunk/range/pace settings; reads recorded deployment addresses.
Attest / demo seller	Loopback ports 8405 / 8406 behind nginx; seller collection key and payment terms; separate systemd units.
Daemon / proxy	Ports 8402 / 8403 on the operator host. Current listen calls omit a bind host. No public exposure should be assumed safe.
Operators	PACTRA_VAULT, PACTRA_REGISTRY, PACTRA_NODE_<label>, PACTRA_KEY_<label>; optional record address and ~/.pactra/pactra.env.

Operations scripts build and copy assets; systemd supervises meter and sellers; nginx supplies routing, compression, response headers and request limits. The public host is designed without daemon operator keys. It does hold seller collection credentials.

Before a fresh deployment, replace every pactra.example origin, configure DNS/TLS and Privy, deploy contracts, regenerate address/ABI fixtures and rebuild both frontends. Existing contract addresses in this copy are historical references, not evidence of a newly branded deployment.

Trust boundaries and limitations

Priority	Finding and consequence	Required response
P0	Unauthenticated daemon/proxy listeners omit a bind host. A reachable caller can act through configured operator authority.	Bind explicitly to a private interface; add caller authentication and node authorization before shared or remote use.
P0	Operator keys can sign raw calls. Contract budgets constrain draws, but final seller payment is not enforced by the vault.	Treat operator execution and key isolation as trusted; add transaction-level reconciliation before claiming payee enforcement.
P1	Vault draw, Gateway settlement and seller collection/delivery are separate. Failed or uncertain settlement may leave funds outside the vault.	Add durable purchase states, idempotency and recovery procedures before automated retries or multi-worker deployment.
P1	Gate publishes only when preflight wouldRelease is false. A simulated acceptance that becomes a receipt refusal can miss publication.	Drive publication from the mined outcome and backfill missing attestations.
P1	Release reservations use a local file and a bounded scan. Crash and multi-process behavior are not fully transactional.	Use coordinated durable reservations; reconcile before releasing locks or rerunning purchases.
P1	ConductRecord embeds an origin and feedback tag in immutable bytecode. Source rename does not update deployed copies.	Deploy a newly configured record contract and treat old identities/records as historical.
P2	Forward-only indexing and retained last-good results can expose stale or incomplete evidence.	Monitor block lag, failed syncs, reconciliation age, truncation and attribution completeness.
P2	The proxy's response path uses text/JSON; arbitrary binary fidelity is not guaranteed.	Define supported payloads and verify content handling for intended clients.

The current reconciliation check is one-sided: Gateway available balance must not exceed cumulative vault releases for live operators. Passing it does not prove exclusive funding, successful delivery, or correct final payee. counterpartyMatching is explicitly unavailable.

Priorities are architecture-review judgments derived from the inspected source, not claims that exploitation was demonstrated. The rename does not attempt unrelated security redesign.

15 / Change and compatibility record

Pactra migration and attribution

Area	Local change
Product identity	Pactra in application copy, titles, descriptions, documentation and generated sharing assets.
Developer identity	SomeshTalligeriDEV in first-party package author fields and updated repository references.
Code / packages	@pactra package scopes, pactra-ui aliases, Pactra React symbols and CSS names; matching imports and filenames updated.
Agent / CLI	pactra_fetch, pactra_spawn, pactra_status; pactra-mcp, pactra-init, pactra-meter, pactra-proxy and pactra-run.
Configuration / operations	PACTRA_* environment names, ~/.pactra/pactra.env, pactra systemd/nginx/script names and explicit example origins.
History	No .git directory was supplied. No Git history was rewritten. The historical first-party deployment commit value is now unknown.
Evidence / dependencies	Historical chain addresses, transaction hashes, dates and numeric evidence retained. Dependency integrity hashes and third-party licensing remain intact.

Breaking configuration changes

This is a coordinated rename, not a compatibility layer. Existing callers must update tool names, environment names, command names, key-file paths and service names together. Protect and migrate existing key material deliberately: never create a new operator key for an existing immutable operator assignment.

pactra.example is a configuration placeholder; GitHub references under SomeshTalligeriDEV/pactra express intended ownership and have not been provisioned or verified. An npm package is not published by changing package.json. Do not run public deployment scripts until these references are configured.

A new maintainer attribution does not establish authorship of every inherited line or alter third-party license obligations. Historical AI-use notes are retained as inherited provenance; the present rename, architecture review and PDF were prepared with Codex.

Validation and release checklist

Check	Result
Builds	Site and console production bundles; daemon, meter, attest, eval, proxy and MCP TypeScript checks passed. MCP bundle and brand generation passed.
Solidity	98 tests passed, 0 failed; Solidity 0.8.28 with Foundry 1.7.1 and the pinned forge-std revision.
Runtime suites	193 passed, 0 failed: daemon 68, meter 39, attest 40, eval 10, MCP 15, proxy 18 and fixtures 3.
Browser	14 route/viewport checks passed: seven routes at desktop 1440 px and mobile 390 px, with no page exceptions or horizontal overflow. Preview/read flows only; no wallet transactions.
Rebrand checks	First-party legacy-name scan clean; package and lockfile names consistent. Ops scripts passed bash syntax checks. Brand assets regenerated.
Document	18 A4 pages; representative pages visually inspected; searchable text, embedded fonts, page numbers and navigation outline. Text bounds verified.
Limits	No public deployment, remote repository change, production login or live payment verification. Build warnings include frontend bundle size and third-party annotations.

What the checks establish

Builds exercise renamed imports, aliases, component symbols, package metadata and generated assets. Automated suites exercise existing contract and runtime behavior within their test environments. A local pass does not validate a production wallet session, a real Pactra domain, public-chain deployment, live seller settlement, or operational key custody.

Before external release

1. Resolve P0 listener and trust-boundary issues for the chosen hosting model.
2. Configure real origins and credentials, deploy and verify matching contracts, and regenerate fixtures.
3. Rehearse owner onboarding, refusal, explicit release, revocation, withdrawal and recovery on the target environment.
4. Verify RPC lag monitoring, snapshots, key backups and incident handling.
5. Verify both frontend bundles in a browser with live and preview states visibly distinct.
6. Measure latency, sustained indexing rate, maximum supported tree size and concurrent-purchase behavior. No numerical SLO, throughput capacity or uptime guarantee is established by this review.

Release status: source rename and architecture deliverables are local artifacts. No public deployment, publication, wallet transaction, or remote Git mutation was performed.

17 / Traceability

Source register and terminology

Evidence path under packages/	Supports
contracts/src/MandateRegistry.sol	Root/child creation, narrowing, ownership, ancestry and revocation.
contracts/src/TreeVault.sol	Treasury, draw bounds, tumbling windows, releases and headroom.
contracts/src/ConductRecord.sol	Identity binding, feedback derivation, immutable record origin.
daemon/src/{gate,fetch,settle,record}.ts	Receipt handling, publication gate, payment order, Gateway and EIP-3009 integration.
daemon/src/{config,keyfile,released,server,main}.ts	Keys, release reservations, environment variables and listener exposure.
mcp/src/server.ts; proxy/src/{proxy,main,run}.ts	Tool schemas, session node, proxy and process-wrapper behavior.
meter/src/{sync,ledger,snapshot,reconcile,server}.ts	Event reductions, persistence, query API and reconciliation limits.
attest/src/{main,server,collect}.ts	Seller startup, payment challenge/verification and collection.
console/src/{App,lib/mandate,lib/wallet}.tsx or .ts	Owner routes, signing actions and preview wallet mode.
site/src/App.tsx; fixtures/src/*.gen.ts	Public routes and supplied historical evidence.
../ops/{nginx,systemd,bin}/	Public-host routing, service processes and release scripts.

Working glossary

Mandate: immutable spending authority for a node. Root: the owner's top-level mandate and treasury scope. Operator: private-key holder permitted to draw for a node. Tranche: one amount released from the vault. Refusal: recorded rejection of a policy-bound draw. Release: owner-signed exception. Headroom: aggregate remaining authority, subject to per-purchase constraints. Attestation: chain-derived published evidence, or a separately sold snapshot when referring to the HTTP service.

x402 is the HTTP payment challenge flow used by the implementation. EIP-3009 supplies a signed token-transfer authorization. ERC-8004 supplies external identity and reputation registries. This document describes how the repository uses these interfaces; it does not independently certify conformance to their specifications.